

UNITY FACTORY AUTOMATION SIMULATION

ProjectF

기술 소개서

대규모 자동화 시뮬레이션을 위한
상태 분리 · 배치 처리 · 스트리밍 설계

한택근 | Client Programmer

개인 프로젝트 | Repository Snapshot 2026.09

Unity 6000.4 · C# · URP · Burst · BatchRendererGroup

핵심 기술 명제

모든 오브젝트를 항상 GameObject 로 유지하지 않는다. 논리 상태를 권위 데이터로 두고, 로드 범위·백그라운드 시뮬레이션·가상 렌더링이 같은 상태를 각자의 비용 구조에 맞게 소비하도록 설계했다.

프로젝트 정의

ProjectF 는 자원 채취, 가공, 운송, 자동화, 영역 확장을 하나의 연속된 시뮬레이션으로 연결하는 Unity 기반 팩토리 자동화 프로젝트다.

개발 형태	1 인 개발 · 게임플레이/시스템/도구 전반 직접 구현
엔진	Unity 6000.4.0f1 · Universal Render Pipeline 17.4.0
핵심 패키지	Burst 1.8.28 · Collections 6.4.0 · Mathematics 1.3.3
저장 포맷	PF_SAVE 매직 헤더를 사용하는 버전형 바이너리 포맷 · 현재 버전 43

해결해야 했던 세 가지 구조적 문제

1. 컨베이어·설치물 수가 늘어날수록 Tick, Transform 계산, Draw Call 비용이 함께 증가한다.
2. 플레이어가 보지 않는 청크에서도 생산과 운송의 논리적 시간은 계속 진행되어야 한다.
3. 실시간 상태, 언로드된 상태, 저장 파일의 상태가 중복되거나 소실되지 않아야 한다.

설계 방향

문제 축	선택한 접근	대표 구현
업데이트 비용	개별 Update 대신 중앙 스케줄링	MapObjectTickManager
렌더링 비용	논리 이동과 표현 분리, 공간 배치	VirtualConveyorBeltRenderer
월드 규모	청크 스트리밍 + 비활성 영역 시뮬레이션	BlockStateStore / BackgroundSimulator
지속성	버전형 바이너리 직렬화와 명시적 복원	SaveGameBinarySerializer

전체 아키텍처

ProjectF 의 중심은 '라이브 오브젝트'가 아니라 '권위 상태'다. 화면과 가까운 영역만 라이브로 유지하고, 나머지는 저장 상태를 기반으로 진행시킨다.

ProjectF 상태·시뮬레이션·표현 아키텍처

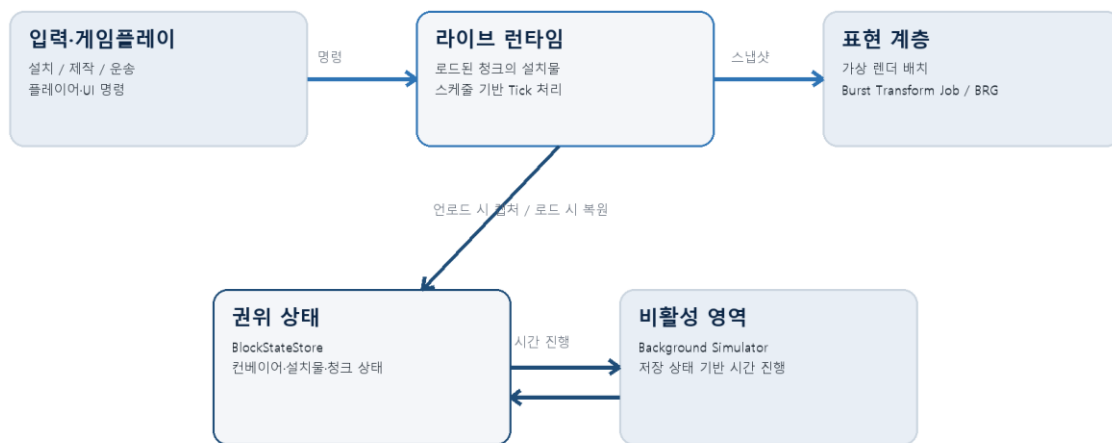


그림 1. 입력, 라이브 런타임, 권위 상태, 백그라운드 시뮬레이션, 표현 계층의 관계

아키텍처 원칙

- 단일 권위 상태:** 청크 좌표와 설치물 앵커를 기준으로 논리 상태를 저장해 중복 소유를 줄인다.
- Live / Virtual 분리:** 로드된 영역은 MonoBehaviour 가 담당하고, 비활성 영역은 직렬화 가능한 상태로 계산한다.
- 스케줄 기반 실행:** 모든 객체가 임의의 Update 를 갖는 대신, 간격별 버킷과 쿼터로 작업을 분산한다.
- 표현의 후행성:** 렌더링은 논리 상태의 결과를 소비하며, GPU 경로 실패 시에도 기능이 유지되는 대체 경로를 둔다.

확장성의 기준

객체 수가 증가할 때 비용이 '컴포넌트 수 × 프레임'으로 선형 누적되지 않도록, 상태·업데이트·표현의 수명주기를 각각 통제한다.

컨베이어 시뮬레이션과 렌더링 최적화

팩토리오의 컨베이어 최적화 방식을 연구하고 AI 를 설계 보조 도구로 활용해, 대량 아이템 이동을 개별 GameObject 중심 구조에서 데이터·배치 중심 구조로 옮겼다.

컨베이어 처리 파이프라인



핵심: 논리 이동과 시각 표현을 분리하고, 반복 할당 대신 재사용 가능한 버퍼와 배치를 사용한다.

그림 2. 컨베이어의 논리 업데이트부터 GPU 표현까지 이어지는 처리 단계

구현 포인트

8. MapObjectTickManager 는 Tick 간격별 버킷과 쿼터·커서를 사용해 같은 프레임에 작업이 몰리는 현상을 줄인다.
9. ConveyorItemTransformJobProcessor 는 Persistent NativeArray 를 2 의 거듭제곱 단위로 확장·재사용하고, IJobParallelFor 로 행렬과 셀 좌표를 계산한다.
10. VirtualConveyorBeltRenderer 는 Mesh·Material·Layer·Submesh·UV 와 공간 셀을 기준으로 인스턴스를 묶어 렌더 호출을 구성한다.
11. VirtualRenderBatchRendererGroupBackend 는 GPU 버퍼와 BatchRendererGroup 을 사용하며, 셰이더나 플랫폼이 맞지 않으면 기존 렌더러로 복귀한다.

결과와 검증 기준

구조적 결과

아이템의 이동 상태와 시각 오브젝트의 생명주기를 분리했고, CPU Transform 계산과 GPU 제출을 배치 단위로 다룰 수 있게 했다. ConveyorRuntimeDiagnostics 는 라이브/저장 아이템 수, 중복 상주, 잘못된 레인·연결을 점검한다.

주의: 저장소에는 동일 조건의 전·후 정량 벤치마크 기록이 없어 FPS-Draw Call 개선률은 기재하지 않았다.

철도 경로 탐색과 자동 운행

철도는 단순한 선형 이동이 아니라, 연결 그래프·열차 편성·역 정차·연료·선로 점유가 동시에 영향을 주는 상태 기반 시스템으로 설계했다.

철도 자동 운행: 경로 데이터에서 상태 머신까지



노선 변경 → 그래프 버전 증가 → 캐시 무효화 → 경로 재계산 → 운행 상태 전이

그림 3. 선로 샘플링, 그래프 구성, 경로 탐색, 자동 운행 상태 머신

경로 데이터

12. Railroad 는 렌더된 중심선 샘플과 누적 거리를 캐시해 위치와 접선을 반복 계산하지 않는다.
13. AutoDriveRoutePlanner 는 선로 연결을 노드와 간선으로 구성하고, 우선순위 큐 기반 탐색에 역주행·급회전 비용을 반영한다.
14. 연결 그래프 버전이 변하면 관련 캐시를 무효화해 과거 노선의 경로를 재사용하지 않는다.

운행 상태와 복원

15. Idle, Planning, Moving, Docking, WaitingAtStation, WaitingForFuel, WaitingForPath, WaitingForClearTrack, Arrived 상태를 명시적으로 분리한다.
16. 연결된 차량은 하나의 편성 그래프로 수집하고, 대표 컨트롤러와 연결 리비전을 통해 구성을 추적한다.

17. 현재 선로 샘플, 목적지, 자동 운행 상태는 저장 데이터에 포함되어 청크 복원 이후에도 운행 맥락을 이어간다.

설계 효과

경로 계산, 실제 이동, 정차·대기 조건을 분리해 디버깅 지점을 명확히 했고, 선로 변경 시 캐시 갱신 범위를 추적할 수 있게 했다.

청크 스트리밍과 백그라운드 시뮬레이션

플레이어 주변의 세부 오브젝트만 유지하면서도 멀리 있는 공장이 멈추지 않도록, 청크 수명주기와 시뮬레이션 수명주기를 분리했다.

언로드부터 복원까지

18. TerrainChunkStreamingScheduler 가 생성·언로드 요청을 큐와 HashSet 으로 중복 제거하고, 프레임별 예산으로 처리한다.
19. 언로드 시 블록·자원·설치물·컨베이어·열차 상태를 BlockStateStore 에 캡처한다.
20. 비활성 청크는 InstallationBackgroundSimulator 가 직렬화된 상태를 대상으로 생산·입출력·운송 시간을 진행한다.
21. 로드 시 의존성이 큰 설치물부터 순서와 예산을 적용해 복원하고, 컨베이어 아이템·열차 샘플·자동 운행 상태를 다시 연결한다.

안정성을 위한 경계

경계	보호 장치	의도
로드/언로드 흔들림	서로 다른 로드·언로드 반경	경계에서 반복 생성 방지
중복 요청	Queue + HashSet	같은 청크의 중복 처리 방지
프레임 스파이크	코루틴과 프레임별 복원 예산	작업량 분산
상태 중복	라이브/저장 상주 진단	아이템 이중 소유 탐지

핵심 불변식

한 시점의 아이템·설치물 상태는 라이브 월드와 저장 상태 중 하나가 권위를 가져야 한다. 스트리밍은 이 권위를 넘기는 과정이며, 진단 코드는 중복 상주와 연결 손상을 검사한다.

저장 포맷과 데이터 원본

저장은 단순한 씬 직렬화가 아니라, 월드의 논리 상태를 다시 구성하기 위한 계약으로 다룬다.

버전형 바이너리 저장

22. SaveGameBinarySerializer 는 PF_SAVE 매직 헤더와 버전 번호를 먼저 기록하고, 지원 범위를 벗어난 파일을 거부한다.
23. 임시 파일에 쓰기를 완료한 뒤 기존 파일을 교체해, 쓰기 도중 완성되지 않은 데이터가 최종 경로에 남는 시간을 줄인다.
24. 지형 시드, 월드 시간, 자원·설치물, 컨베이어 레인, 열차 자동 운행, 유체, 플레이어·인벤토리·제작 큐를 명시적으로 직렬화한다.

데이터 진실 원본

데이터	진실 원본	런타임 소비 방식
아이템	ItemDefinition ScriptableObject	ID-에너지·유체·제작 속성 참조
제작 트리	crafting_tree.bytes 바이너리	CraftingTreeRuntime 이 역색인 Dictionary 구성
월드 상태	BlockStateStore + SaveGameData	로드 범위에 따라 라이브 오브젝트로 복원

유체 네트워크

파이프와 탱크는 연결 마스크와 네트워크 탐색을 사용하고, 토폴로지 버전과 연결 캐시로 불필요한 재탐색을 줄인다. 지하 파이프는 원격 좌표와 쌍 연결을 별도로 관리하며, 펌프는 출력 네트워크의 용량·충전 비율을 기준으로 전달 대상을 계산한다.

데이터 설계의 목적

편집용 원본, 런타임용 산출물, 저장 상태를 구분해 동기화 방향을 명확히 하고, 포맷 변경이 어떤 복원 경로에 영향을 주는지 코드에서 드러나게 한다.

진단 도구와 AI 활용 개발

AI 는 코드를 대신 책임지는 도구가 아니라, 복잡한 최적화 대안을 빠르게 탐색하고 구현 범위를 넓히는 협업 도구로 활용했다.

검증을 위한 하네스

25. ConveyorRuntimeDiagnostics 는 로드·저장 블록의 아이템 총량, 중복 상주, 잘못된 레인·링크를 감사한다.
26. MapObjectTickManager 의 ProfilerMarker·Counter 는 Tick 비용과 분산 상태를 관찰할 수 있는 진입점을 제공한다.
27. 별도 WinForms 기반 MapObjectProfilerTool 은 localhost TCP 로 FPS, 프레임 시간, Tick 루프, 컨베이어 상태 지표를 수집한다.

AI 활용 루프

28. 문제 정의: '오브젝트 수가 늘수록 어떤 비용이 누적되는가'를 상태·연산·표현으로 분해한다.
29. 대안 탐색: 팩토리오식 데이터 중심 처리, Unity Jobs/Burst, 공간 배치, BRG 적용 가능성을 AI 와 비교한다.
30. 구현·리뷰: AI 가 제안한 구조를 프로젝트의 권위 상태, 체크 수명주기, GC 제약에 맞게 수정하고 중복 코드를 제거한다.
31. 검증: 진단 API 와 프로파일러로 상태 불변식과 비용을 확인하고, 실패 가능한 GPU 경로에는 폴백을 둔다.

AI 활용으로 확장한 구현 범위

혼자서 진입하기 어려웠던 Burst Job, BatchRendererGroup, 그래프 기반 경로 탐색, 백그라운드 상태 시뮬레이션을 작은 검증 단위로 나누어 구현할 수 있었다. 최종 설계 선택과 코드 통합, 오류 판단은 개발자가 직접 수행했다.

코드 맵과 다음 검증 과제

핵심 시스템은 역할별 파일로 분리되어 있으며, 아래 경로가 기술 검토의 시작점이다.

영역	대표 코드	검토 포인트
컨베이어	VirtualConveyorBeltRenderer.cs ConveyorItemTransformJobProcessor.cs	배치 키·버퍼 재사용·폴백
렌더 백엔드	VirtualRenderBatchRendererGroupBackend.cs	GPU 버퍼·컬링·호환성
Tick	MapObjectTickManager.cs	버킷·쿼터·Profiler
철도	SteamTrain.cs / Train.cs / Railroad.cs	그래프·경로·상태 머신
월드 상태	BlockStateStore.cs InstallationBackgroundSimulator.cs	권위 상태·비활성 시뮬레이션
스트리밍	TerrainChunkStreamingScheduler.cs TerrainGenerator.ChunkPersistence.cs	예산·복원 순서·중복 요청
저장	SaveGameBinarySerializer.cs SaveGameData.cs	포맷 버전·복원 계약
진단	ConveyorRuntimeDiagnostics.cs Tools/MapObjectProfilerTool	상태 감사·성능 지표

다음 우선순위

32. 동일 맵·아이템 수를 고정한 자동 벤치마크를 만들어 CPU 시간, 배치 수, GC Alloc 을 전·후 비교한다.
33. SteamTrain 의 경로 계획·운행 상태·편성 제어 책임을 점진적으로 분리해 테스트 가능한 경계를 넓힌다.
34. 컨베이어와 스트리밍 시나리오를 재현하는 테스트 하네스를 정식화하고 CI 에서 저장·복원 불변식을 검사한다.

35. 의존 관계가 안정된 시스템부터 namespace 와 asmdef 를 도입해 컴파일 경계를 명확히 한다.

정리

ProjectF 의 기술적 핵심은 기능의 수가 아니라, 대규모 월드에서 상태의 권위와 비용의 발생 지점을 통제하는 구조다. 컨베이어·철도·스트리밍·저장은 이 원칙을 서로 다른 문제에 적용한 결과다.